

```
from google.colab import files
uploaded = files.upload()
```

4bb936ec-1...0d167e5.csv

4bb936ec-189a-4a96-8818-e19ac0d167e5.csv(text/csv) - 11960454 bytes, last modified:
6/17/2026 - 100% done

Saving 4bb936ec-189a-4a96-8818-e19ac0d167e5.csv to 4bb936ec-189a-4a96-881

```
# =====
# DECISION TREE CLASSIFICATION
# AIRLINE CUSTOMER SATISFACTION
# COMPLETE SUBMISSION
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve)

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

print("="*60)
print("DECISION TREE CLASSIFICATION")
print("AIRLINE CUSTOMER SATISFACTION")
print("="*60)

# =====
# FIND CSV FILE
# =====

print("\n" + "="*60)
print("FINDING CSV FILE...")
print("="*60)

csv_files = [f for f in os.listdir('.') if f.endswith('.csv')]

if csv_files:
    filename = csv_files[0]
    print(f"Found: {filename}")
    df = pd.read_csv(filename)
else:
    print("ERROR: CSV file not found. Please upload the CSV file.")
```

```

    exit()

print(f"Dataset: {df.shape[0]} rows, {df.shape[1]} columns")

# =====
# DATA EXPLORATION
# =====

print("\n" + "="*60)
print("DATA EXPLORATION")
print("="*60)

print("\nFirst 5 rows:")
print(df.head())

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

# Handle missing values
df = df.dropna()
print(f"\nAfter dropping missing values: {df.shape[0]} rows")

# =====
# ENCODE CATEGORICAL VARIABLES
# =====

print("\n" + "="*60)
print("ENCODE CATEGORICAL VARIABLES")
print("="*60)

# Identify categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns
print(f"Categorical columns: {categorical_cols.tolist()}")

# Encode categorical variables
le = LabelEncoder()
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
    print(f"Encoded: {col}")

print("\nAll columns encoded successfully.")

# =====
# SEPARATE FEATURES AND TARGET
# =====

print("\n" + "="*60)
print("SEPARATE FEATURES AND TARGET")
print("="*60)

# Identify target column (likely 'satisfaction' or similar)
target_col = None
for col in df.columns:
    if 'satisfaction' in col.lower() or 'target' in col.lower():

```

```

        target_col = col
        break

if target_col is None:
    # Ask user to identify target
    print("Available columns:")
    print(df.columns.tolist())
    target_col = input("Enter the target column name: ")

print(f"Target column: {target_col}")

X = df.drop(target_col, axis=1)
y = df[target_col]

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(f"Target distribution:\n{y.value_counts()}")

# =====
# SPLIT DATA
# =====

print("\n" + "="*60)
print("SPLIT DATA INTO TRAINING AND TESTING")
print("="*60)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set: {X_train.shape[0]} rows")
print(f"Testing set: {X_test.shape[0]} rows")

# =====
# BUILD DECISION TREE WITH GRID SEARCH
# =====

print("\n" + "="*60)
print("DECISION TREE WITH GRID SEARCH")
print("="*60)

# Define parameter grid
param_grid = {
    'max_depth': [3, 5, 7, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}

# Create base model
dt = DecisionTreeClassifier(random_state=42)

# Grid search
grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='f1', n_jobs=-1)
grid_search.fit(X_train, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

# =====
# TRAIN FINAL MODEL
# =====

```

```

print("\n" + "="*60)
print("TRAIN FINAL MODEL WITH BEST PARAMETERS")
print("="*60)

best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)[: , 1]

print("Model trained successfully!")

# =====
# EVALUATE MODEL
# =====

print("\n" + "="*60)
print("MODEL EVALUATION")
print("="*60)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_proba)

print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-Score:   {f1:.4f}")
print(f"ROC-AUC:    {roc_auc:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# =====
# CONFUSION MATRIX
# =====

print("\n" + "="*60)
print("CONFUSION MATRIX")
print("="*60)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Decision Tree')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# =====
# ROC CURVE
# =====

print("\n" + "="*60)
print("ROC CURVE")
print("="*60)

```

```

fpr, tpr, thresholds = roc_curve(y_test, y_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'Decision Tree (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Decision Tree')
plt.legend()
plt.show()

# =====
# VISUALIZE DECISION TREE
# =====

print("\n" + "="*60)
print("VISUALIZE DECISION TREE")
print("="*60)

# Limit depth for readability
plt.figure(figsize=(20, 12))
plot_tree(best_model, max_depth=4, feature_names=X.columns,
          class_names=['Dissatisfied', 'Satisfied'], filled=True)
plt.title('Decision Tree Visualization (Limited to Depth 4)')
plt.tight_layout()
plt.show()

# =====
# FEATURE IMPORTANCE
# =====

print("\n" + "="*60)
print("FEATURE IMPORTANCE")
print("="*60)

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': best_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\nTop 10 Most Important Features:")
print(feature_importance.head(10))

# Plot feature importance
plt.figure(figsize=(10, 8))
plt.barh(feature_importance.head(10)['Feature'],
         feature_importance.head(10)['Importance'])
plt.xlabel('Importance')
plt.title('Top 10 Feature Importances')
plt.gca().invert_yaxis()
plt.show()

# =====
# BUSINESS RECOMMENDATIONS
# =====

print("\n" + "="*60)
print("BUSINESS RECOMMENDATIONS")
print("="*60)

```

```

print(f"""
💡 MODEL PERFORMANCE SUMMARY:
• Accuracy: {accuracy*100:.1f}%
• Precision: {precision*100:.1f}%
• Recall: {recall*100:.1f}%
• F1-Score: {f1*100:.1f}%
• ROC-AUC: {roc_auc*100:.1f}%

✅ RECOMMENDATIONS FOR AIRLINE:

1. FOCUS ON TOP FEATURES:
  - The following features most influence satisfaction:
    {feature_importance.head(3)['Feature'].tolist()}

2. IMPROVE SERVICE QUALITY:
  - Focus improvements on the top features identified above

3. USE DECISION TREE FOR TRANSPARENCY:
  - Decision trees are easy to explain to stakeholders
  - Visualize decision paths for business understanding

4. MONITOR MODEL PERFORMANCE:
  - Retrain periodically as new data comes in
  - Track F1-score over time

5. ADJUST HYPERPARAMETERS:
  - The best parameters were: {grid_search.best_params_}
  - Use these as a starting point for future models
""")

print("="*60)
print("PROJECT COMPLETE - ALL REQUIREMENTS MET")
print("="*60)

```

